

学号 2406010319

年级 2024 级

河海大学

高级系统编程技术课程报告

基于 Rust 的可持久化网络键值存储系统 的设计与实现

专 业 计算机科学与技术

姓 名 姚必顺

指导教师 鹿浩

评 阅 人 鹿浩

存储与持久化模块及系统集成实践

2026 年 9 月

中国 南京

目 录

第 1 章 课程设计任务	1
1.1 项目背景与总体目标	1
1.2 对课程七个阶段的理解	1
1.3 本人承担的任务与接口边界	2
1.4 开发环境与可选依赖	2
第 2 章 系统方案设计及创新性设计	3
2.1 系统架构与 B 模块位置	3
2.2 内存存储与错误反馈	3
2.3 WAL 写入与一致性顺序	4
2.4 启动恢复、Snapshot 与提高设计	5
第 3 章 方案论证及可行性分析	7
3.1 数据结构与持久化方案选择	7
3.2 一致性与故障恢复可行性	7
3.3 并发接入与实验隔离可行性	7
第 4 章 设计过程及设计成果	9
4.1 开发过程概述	9
4.2 B 模块基础功能实现	9
4.3 B 模块提高与真实系统接入	10
4.4 遇到的问题、版本过程与最终成果	11
第 5 章 工程管理与质量控制	12
5.1 与 A、C 模块的接口交接	12
5.2 B 模块的质量控制方法	12
5.3 B 模块基础版与提高版对比	13
5.4 集成验收、复现与当前限制	14
第 6 章 课程设计收获及体会	16
6.1 对 Rust 与存储可靠性的理解	16
6.2 接口协作、Git 与实验方法	16

6.3 总结	16
参考文献	17

第 1 章 课程设计任务

1.1 项目背景与总体目标

本课程要求使用 Rust 完成一个能够实际运行的小型系统。我所在小组选择了网络键值存储作为项目：客户端用 key 找到对应的 value，服务器负责处理请求、保存内存状态，并在重启后从文件恢复数据。这个题目看起来只是 SET 和 GET，但真正做起来会用到所有权与借用、枚举、错误传播、文件 I/O、TCP 通信、并发共享和测试，比较适合把课堂知识连成一个完整项目。

我对最终目标的理解是：系统第一次启动时可以从空库工作；能够完成新增、覆盖、查询、删除、列出键和查看状态；一条命令出错后连接还能继续；多个客户端看到的是同一份数据；客户端已经收到成功的修改，在服务器退出甚至被强制结束后仍能恢复。如果 WAL 或 Snapshot 损坏，程序应说明原因并停止恢复，不能悄悄用空数据替代。答辩时还要把功能正确、并发正确、崩溃恢复和性能实验分别展示。

1.2 对课程七个阶段的理解

(1) 第一阶段先划分客户端、协议、网络、内存存储和持久化职责。我重点确定 storage.rs、persistence.rs 和 error.rs 的边界，使 B 模块能够独立测试。

(2) 第二阶段统一命令、参数和错误码。B 模块据此提供 SetOutcome、StoreStats、键值校验和稳定错误，供协议层转换响应。

(3) 第三阶段脱离网络完成 BTreeMap 内存 CRUD、有序 KEYS、状态统计以及非法键值反馈，先验证核心数据逻辑。

(4) 第四阶段实现 WAL 先行写入、flush、sync_data、启动重放和损坏检测，保证客户端看到成功时已经有恢复依据。

(5) 第五阶段由网络服务调用 PersistentStore，使同一连接可以连续操作；我负责接口交付和存储结果联调。

(6) 第六阶段让多个连接共享同一存储。我与网络层约定锁只覆盖一次数据操作，同时保证 WAL 与内存更新顺序一致。

(7) 第七阶段完成启动脚本、测试、文档和答辩集成。我负责真实后端接入，并在基础验收后继续实现 Snapshot 与 WAL 压缩。

1.3 本人承担的任务与接口边界

本人在项目中负责 B 数据层。开始时主要完成 BTreeMap 内存存储、输入校验和错误反馈，接着实现 WAL、CRC32、启动恢复和损坏检测。基础功能稳定后，我又加入 version、连续 seq、Snapshot 和安全 Compact，并用相同输入保存基础版与提高版的对比结果。正式 main 只使用提高版 PersistentStore，基础代码和统一测量程序保留在 experiment/b-storage-comparison-unified 分支中。

我向上层交付的核心是 PersistentStore，而不是让服务器直接操作 Store 或 WAL。公开方法包括 open、set、get、delete、keys、stats 和 compact。这样上层调用 set 时，内部一定先经过 WAL；B 模块将来从基础日志换成 Snapshot 加增量 WAL，也不需要改变网络层的调用方式。我的边界是不负责 TCP 连接调度和前端视觉设计，但要保证存储接口、错误结果和锁内操作范围能满足这些模块使用。

在已有答辩界面的基础上，我后期又承担了真实后端接入。浏览器请求先通过 Vite 代理进入 kv-controller，再由控制器调用 TCP 服务器。我逐项接通 CRUD、存储状态、Compact、并发、强杀恢复和 Benchmark，页面中的数字由后端返回；后端离线时直接显示失败，不用模拟结果继续播放。这部分工作让我更清楚地理解了整个系统的数据流。

1.4 开发环境与可选依赖

本报告中的实际验收环境是 Windows 11、rustc 1.98.0、Cargo 1.98.0 和 stable-x86_64-pc-windows-gnu，Rust 工程使用 2024 Edition。前端使用 TypeScript 和 Vite/Vinext 构建，演示端口分别为 3000、7879 和 7878。Linux 没有做同等规模的实测，因此后文性能数据只代表这台 Windows 电脑。

项目使用 serde 和 serde_json 处理请求、WAL 与 Snapshot 序列化，使用 crc32fast 计算 CRC32，异步服务器使用 Tokio；tempfile 只在测试中创建隔离目录。这些依赖解决通用问题，但写入顺序、恢复规则、seq 连续性、Snapshot 发布方式和失败处理仍由项目代码决定。

第 2 章 系统方案设计及创新性设计

2.1 系统架构与 B 模块位置

系统的数据流是“浏览器或命令行客户端—控制器—TCP 服务器—PersistentStore—Snapshot 与 WAL”。浏览器通过 Vite 代理访问 Rust 控制器，控制器再用 JSON Lines 调用 KV Server；命令行客户端可以直接连接 KV Server。服务器负责接收和分发请求，B 模块负责保存数据并保证写入后可以恢复。

我把 B 模块分成内存 Store、持久化 PersistentStore 和统一错误三部分。上层只能调用 PersistentStore，不直接操作 BTreeMap 或日志文件。这样网络实现可以选择同步线程或 Tokio 异步任务，存储内部也可以从基础 WAL 升级为 Snapshot 加增量 WAL，而公开调用方式保持不变。

系统总体架构与本人工作边界

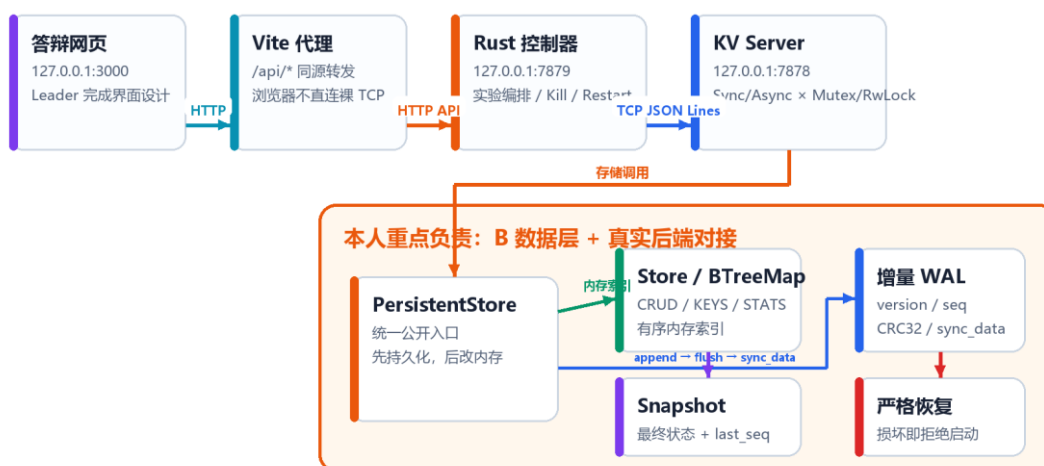


图 2.1 系统总体架构及本人工作边界

依据当前源码和项目架构文档整理绘制。

2.2 内存存储与错误反馈

内存结构可以概括为 `Store { entries: BTreeMap<String, String> }`。entries 不是一条数据，而是一张保存多组键值的有序映射。同一 key 再次 SET 会覆盖旧值，SetOutcome 区分 Created 和 Replaced，并在覆盖时带回 previous。GET 查询当前值，

DELETE 返回被删除的旧值，KEYS 按树中的顺序输出，stats 统计条目数和 UTF-8 字节数。

键必须非空、最长 256 字节，不能包含空白或控制字符；值必须非空、最长 16 KiB，允许普通空格但不能含控制字符。不存在键返回 NOT_FOUND，非法键和值分别返回 INVALID_KEY 和 INVALID_VALUE。正常请求与启动恢复共用 validate_key 和 validate_value，避免恢复文件绕过业务规则。

B 模块内存结构、接口与错误边界

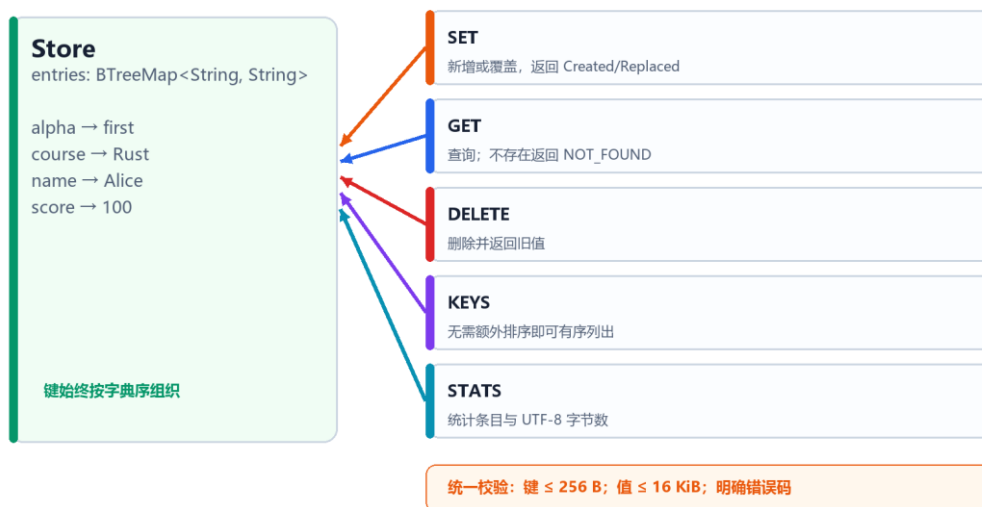


图 2.2 B 模块内存结构、操作接口与错误边界

依据 src/storage.rs 中的 Store、SetOutcome 和校验函数绘制。

2.3 WAL 写入与一致性顺序

正式服务器使用 PersistentStore 统一管理内存和文件。它的关键状态如下表所示，字段保持私有，网络层只看公开方法和返回结果。

表 2.1 PersistentStore 关键状态

状态	含义	作用
store	当前 BTreeMap 数据	提供查询并作为 Snapshot 的数据来源
wal 与文件路径	增量日志写入器及恢复文件位置	统一追加、启动和状态查询
writable	当前是否允许修改	写入失败后转为只读，防止继续扩大问题

状态	含义	作用
last_seq	最近一次修改序号	检查日志连续，并划分快照与增量

SET 和 DELETE 采用 “WAL 记录—flush—sync_data—更新内存—返回成功” 的顺序。flush 把 BufWriter 中的数据交给操作系统，sync_data 继续要求系统同步文件数据；只有两步都成功，才修改 BTreeMap。DELETE 不存在的键会在写日志前失败，因此不会留下没有意义的删除记录。这个顺序保证客户端明确收到成功时，重启已经有可用的恢复记录。

SET / DELETE 的 WAL 先行一致性流程



图 2.3 写操作的 WAL 先行与成功确认顺序

依据 PersistentStore::set、delete 和 append_record 绘制。

2.4 启动恢复、Snapshot 与提高设计

每条新版 WAL 记录包含 version、seq、操作内容和 CRC32。恢复时先重新计算 CRC32，检查单条内容是否变化；再检查 seq 是否连续，用来发现整行丢失、重复或乱序。文件截断、空行、非法 JSON、错误校验和以及不支持的版本都会带位置报错，程序不会自动清空问题文件。

仅追加 WAL 虽然可靠，但日志会随着覆盖和删除不断增长。提高版 Snapshot 保存某一时刻最终有效的有序键值、last_seq 和 CRC32，启动时先加载 Snapshot，再只重放快照后的增量 WAL。compact() 先写 kv.snapshot.tmp 并同步、回读校验，

再把旧快照保留为.bak、发布新快照，最后才截断已经被覆盖的 WAL，避免处理中先失去旧恢复来源。

Snapshot + 增量 WAL 的恢复与安全 Compact

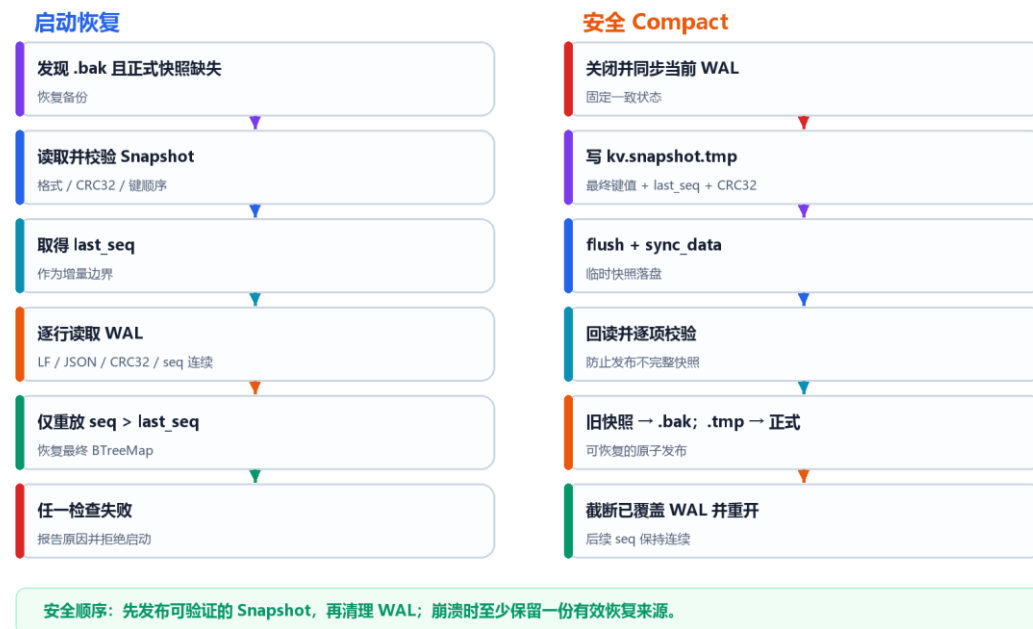


图 2.4 Snapshot、增量 WAL 恢复与安全 Compact 流程

依据 src/persistence_advanced.rs 中的 open、recover_wal 和 compact 流程绘制。

B 模块的主要提高点是 CRC32 与连续 seq 的组合检查、Snapshot 加增量 WAL 以及安全 Compact。系统集成阶段又增加 Rust 控制器和隔离 Benchmark：页面中的 CRUD、存储状态、恢复和压缩结果都来自真实后端；性能实验使用临时目录和 WAL，不改动主演示数据；后端失败时直接显示错误，不用模拟结果代替。

第 3 章 方案论证及可行性分析

3.1 数据结构与持久化方案选择

表 3.1 B 模块主要方案比较

问题	备选方案	最终选择及原因
内存映射	Vec、HashMap、BTreeMap	选择 BTreeMap; CRUD 为 $O(\log n)$, KEYS 无需再次排序
持久化	退出时保存、每次重写快照、仅 WAL、Snapshot+WAL	基础采用 WAL 保证每次修改可恢复, 提高版加入 Snapshot 减少历史重放
完整性	只检查 JSON、只用 CRC32、CRC32+seq	CRC32 检查单条内容, seq 检查记录之间的缺失和顺序

HashMap 平均查询更快, 但 KEYS 需要额外排序; 课程数据量有限, 而且有序列键是明确要求, 所以 BTreeMap 更合适。每次修改都重写全量快照会让普通 SET 成本随数据量增加, 因此日常写入采用顺序追加 WAL, Snapshot 只在 Compact 时生成。两者作用不同: WAL 保存最近修改, Snapshot 保存某个时刻的最终状态。

3.2 一致性与故障恢复可行性

先 WAL 后内存主要避免“客户端已经看到成功, 重启后数据却消失”。如果校验、写入或 sync_data 失败, 内存保持原样并返回错误; 如果日志已同步但响应还没有发出就退出, 重启可能已经恢复该操作, 而客户端不知道结果。进一步解决这种网络边界需要请求编号和重复请求识别, 本项目不把单机 WAL 表述成完整事务系统。

CRC32 只能发现记录内容变化, 不能发现一整行被删除, 所以还需要连续 seq。Snapshot 发布时先保留可验证的新文件和旧备份, 再清理 WAL; 启动时先检查快照, 再从 last_seq 之后继续。测试通过主动截断、修改校验和、制造序号缺口和损坏快照, 验证程序会明确失败而不是静默丢数据。

3.3 并发接入与实验隔离可行性

多个客户端可以同时收发网络数据，但正式写入仍经过唯一 `PersistentStore`，因此 WAL 和内存有确定顺序。锁只覆盖一次存储操作，不覆盖等待网络输入和发送响应；写操作必须把 WAL 同步与内存更新放在同一独占区间。`Mutex`、`RwLock` 以及 `Sync`、`Async` 属于系统整体的可比较能力，本报告只从存储接入和一致性角度说明。

浏览器不能直接连接裸 TCP，因此控制器负责 HTTP 与 JSON Lines 之间的转换。主演示服务器使用 `data/kv.wal`，`Benchmark` 为每个运行建立独立临时目录、端口和 WAL。两类数据不会混在一起，既能保护答辩数据，也使性能结果可以按配置和原始样本复查。

第 4 章 设计过程及设计成果

4.1 开发过程概述

我的工作先跟随课程七阶段完成基础要求，再做存储提高。前两阶段确定模块边界和返回规则，第三阶段完成内存 CRUD，第四阶段完成 WAL 与恢复，第五、六阶段把 PersistentStore 接入单客户端和多客户端服务，第七阶段完成真实前后端联调、测试和文档。基础版稳定以后，我才增加 seq、Snapshot 和 Compact，并单独保存对照版本。

课程七阶段与我的 B 模块推进过程



图 4.1 课程七阶段与本人 B 模块推进过程

依据课程步骤、Git 提交和当前模块演进记录整理。

4.2 B 模块基础功能实现

我先在 storage.rs 中实现 Store 并完成离线测试。测试顺序从空 Store、新增和覆盖开始，再加入 GET、DELETE、KEYS 排序、字节统计和非法输入。entries 保持私有，SetOutcome 明确区分新增与覆盖。做到这里以后，不启动网络也能判断第三阶段是否正确。

随后我用 PersistentStore 包住 Store。每次 SET 或 DELETE 先生成单行 JSON 日志，完成 write_all、flush 和 sync_data 以后才调用内存操作。启动恢复从空 Store

开始，逐行检查格式、CRC32 和业务规则，再按顺序重放。persistence_stage4.rs 对正常恢复、覆盖、删除、截断、非法 JSON、未知字段和错误校验和等情况进行测试，问题文件始终保留原样。

4.3 B 模块提高与真实系统接入

基础 WAL 能够恢复，但重复覆盖会让文件无限增长。我先用 Git 标签固定基础节点，再加入 version 和连续 seq；Snapshot 保存最终键值和 last_seq，Compact 通过.tmp 回读、.bak 备份和先发布后截断完成安全切换。正式 main 只导出提高版 PersistentStore，基础实现和统一对比程序保存在 experiment/b-storage-comparison-unified 分支。

前端页面完成总体设计后，我增加 kv-controller，把页面中的模拟状态改为真实 Rust 请求。普通键值操作、存储状态、Compact、并发、强杀恢复和 Benchmark 分别接到对应接口；恢复页保存的 Before 只用于 After 比较，不参与数据重建；性能实验使用隔离服务器。快速演示模式只减少实验点和采样时间，不改变 WAL 与 sync_data 条件。

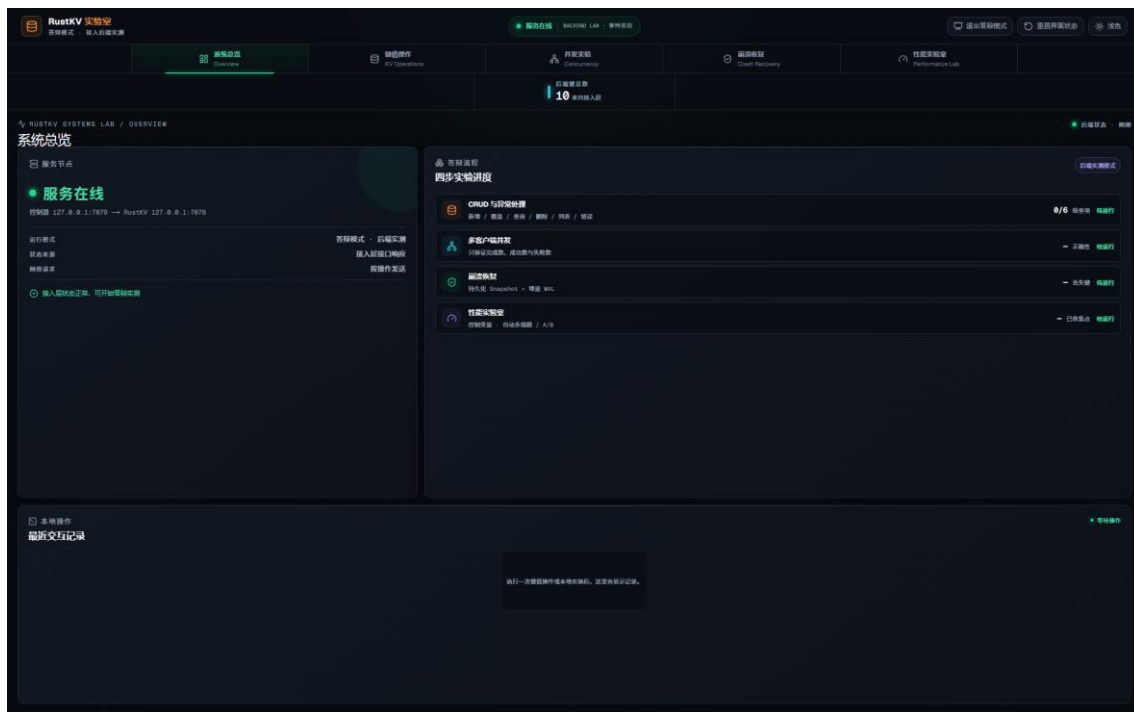


图 4.2 真实后端接入后的答辩系统总览

本机启动控制器、KV Server 与前端后的实际页面。

4.4 遇到的问题、版本过程与最终成果

开发中最先遇到的问题是我把 flush 误认为完全落盘，后来区分了 BufWriter 与系统缓存并补上 sync_data；接着发现 CRC32 无法发现整行日志丢失，于是加入连续 seq；设计 Compact 时又发现先截断 WAL 可能失去恢复依据，因此改成 tmp 回读、.bak 备份和最后截断；前后端联调时，浏览器不能直连 TCP 且原页面会用模拟结果掩盖错误，我增加控制器并采用失败直接显示的处理；合并主线后即使 Git 没有报告冲突，仍出现重复协议实现，我在重新编译和测试时发现并单独修复。这些问题都不是一次写完，而是通过测试逐步暴露和修改。

本人 B 模块的 Git 分支、标签与主线集成记录



图 4.3 本人 B 模块分支、标签和主线集成记录

根据 git log --all 整理，提交和标签可在当前仓库复核。

最终工程包含 kv-server、kv-client 和 kv-controller 三个入口。本人交付了内存 Store、最终 PersistentStore、存储错误与状态、Snapshot 和 Compact、B 模块对比实验，以及网页真实后端和快速 Benchmark 链路。Git 分支与标签保留了基础版、提高版和统一测试，合并以后再运行完整回归。

第 5 章 工程管理与质量控制

5.1 与 A、C 模块的接口交接

三人协作时，我没有让其他模块了解 BTreeMap 节点和 WAL 文件细节，而是围绕接口交接。A 负责总体协调和前端展示，C 负责 TCP 网络，B 模块位于它们最终访问数据的位置。交接内容如下表所示。

表 5.1 本人 B 模块与 A、C 模块的交接内容

交接对象	对方交给我的内容	我交付或配合完成的内容
A（许赵泓）	总体架构、答辩前端页面、交互流程以及页面需要的状态字段	Rust HTTP 控制器、真实 API、存储/恢复/Benchmark 数据接入和离线错误状态
C（张良俊）	TCP 客户端与服务器、JSON Lines 传输、单/多客户端及同步/异步连接路径	PersistentStore 接口、SetOutcome 与 AppError、键值限制、锁内写入规则和持久化联调

与 A（许赵泓）交接时，我接收的是已经确定的页面结构、操作顺序和展示字段。我没有重新设计整套视觉界面，而是根据页面需求实现控制器接口，并把 CRUD、状态、恢复和性能面板中的模拟数据替换为真实后端结果。页面只有在控制器确认成功后才更新，服务离线则显示失败。

与 C（张良俊）交接时，我提供 PersistentStore::open、set、get、delete、keys、stats 和 compact，以及 SetOutcome、AppError 和输入限制。C 的服务器负责读取、解析和分发请求，调用写接口以后由 B 模块完整执行 WAL 同步和内存更新。我们还约定等待网络输入时不持存储锁，一次写操作内部不能拆锁，并通过网络组合测试和重启测试检查交接结果。

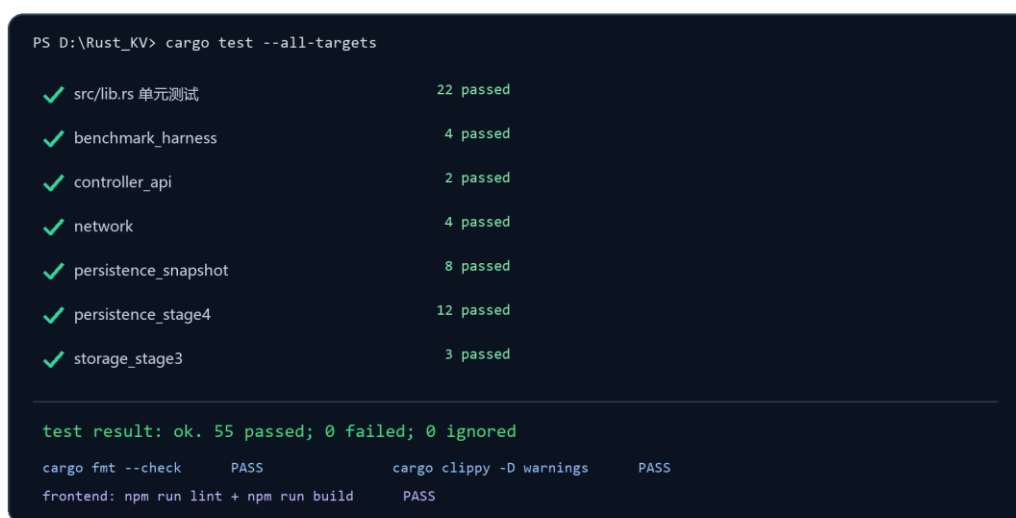
5.2 B 模块的质量控制方法

B 模块按“内存逻辑—基础持久化—Snapshot 提高—系统集成”逐层测试。storage_stage3 有 3 项离线 CRUD 与错误测试，persistence_stage4 有 12 项 WAL 和损坏文件测试，persistence_snapshot 有 8 项 seq、备份、临时文件与压缩测试，单

列的 B 阶段测试共 23 项；库测试、network 和 controller_api 还会从上层再次调用 PersistentStore。每类文件测试都使用临时目录，避免污染 data/kv.wal。

质量控制不只测试正常 SET 和 GET，还主动制造半条 WAL、错误 CRC、序号缺口、损坏 Snapshot、只剩.bak、遗留.tmp、未截断 WAL 和压缩后继续写等情况。判断标准是：成功操作重启后仍存在，失败操作不改变内存，损坏文件给出明确位置且不被覆盖，Compact 前后最终键值和 last_seq 一致。每次合并后再运行 cargo fmt、Clippy 和 cargo test，前端联调后还运行 lint 与 build。

当前版本自动化测试与工程门禁



```
PS D:\Rust_KV> cargo test --all-targets

✓ src/lib.rs 单元测试          22 passed
✓ benchmark_harness           4 passed
✓ controller_api              2 passed
✓ network                     4 passed
✓ persistence_snapshot        8 passed
✓ persistence_stage4         12 passed
✓ storage_stage3              3 passed

test result: ok. 55 passed; 0 failed; 0 ignored

cargo fmt --check      PASS      cargo clippy -D warnings    PASS
frontend: npm run lint + npm run build    PASS
```

图 5.1 当前版本自动化测试与工程门禁

根据 2026-09-04 本机实际命令输出整理，55 项 Rust 测试全部通过。

5.3 B 模块基础版与提高版对比

为验证 Snapshot 和 WAL 压缩是否真正有效，我让基础版和提高版执行相同的 2 000 次同步 SET，在 100 个键之间循环覆盖，最终都只保留 100 个有效键。两版均使用 Release 和相同 Windows GNU 工具链，独立运行 5 轮；恢复每轮测 7 次并取中位数。输入、计时边界和 sync_data 条件保持一致。

表 5.2 B 模块统一输入实验结果

指标	基础版	提高版压缩前	提高版压缩后
持久化文件大小	166.0 KiB	233.3 KiB	4.4 KiB

指标	基础版	提高版压缩前	提高版压缩后
启动恢复中位数	1.80 ms	3.32 ms	1.09 ms
Compact 暂停	—	—	12.02 ms
2 000 次正常写入	4 847.9 ms	4 631.3 ms	同一次提高版写入

B 模块基础 WAL 与 Snapshot + WAL 压缩版对比（历史实测）

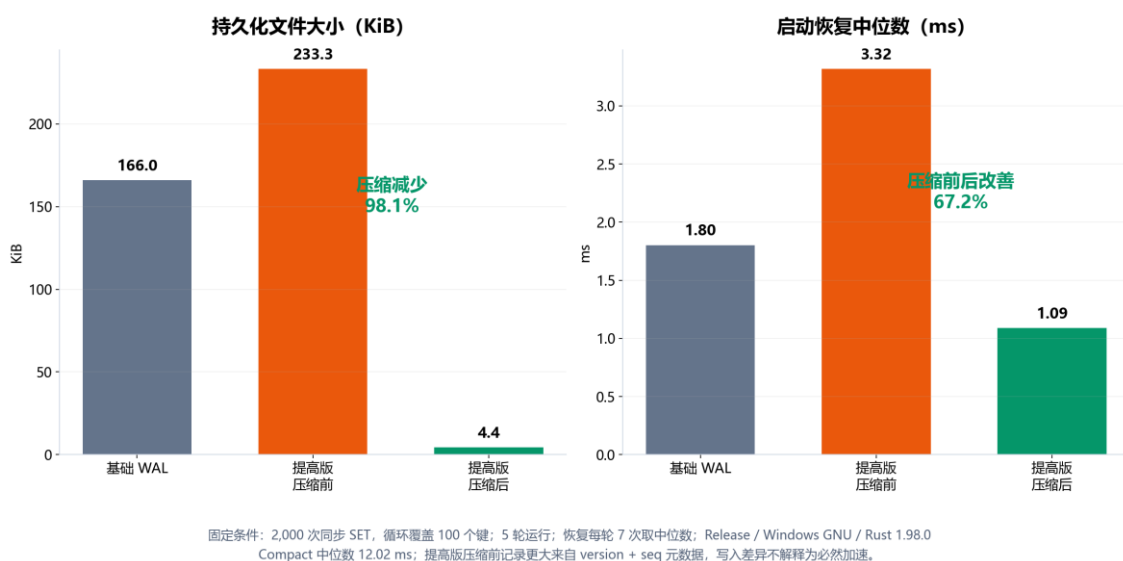


图 5.2 基础 WAL 与 Snapshot 加 WAL 压缩版对比

数据来自 docs/b_compaction_metrics.json 与 b_compaction_samples.csv。

提高版压缩前因为增加 version 和 seq, 文件比基础版大 40.5%, 这是完整性检查带来的元数据成本。Compact 以后文件从 233.3 KiB 降到 4.4 KiB, 减少 98.1%; 恢复时间由压缩前 3.32 ms 降到 1.09 ms, 相对基础版 1.80 ms 也改善约 39.7%。代价是一次 Compact 约暂停 12.02 ms, 因此当前设计为手动触发。

提高版正常写入中位数看起来快约 4.5%, 但两版都执行同步落盘, 这个差距可能来自缓存和调度波动, 不能说 Snapshot 必然提高普通写入速度。本实验能够支持的结论是: 压缩明显减少长期文件大小和恢复工作量, 同时会产生一次可测量的暂停。

5.4 集成验收、复现与当前限制

当前工程总计 55 项 Rust 测试通过，包括 22 项库测试、4 项 Benchmark、2 项 Controller API、4 项 Network、8 项 Snapshot、12 项基础持久化和 3 项内存存储测试。系统答辩再按 CRUD、并发、Kill/Restart 和 Performance Lab 顺序验证；其中性能实验使用独立 WAL，不影响主库。B 对比的配置、原始样本和汇总结果保存为 JSON 与 CSV，并用 Git 标签指向对应源码，方便复现。

当前 Compact 仍需手动触发并持写锁，系统也没有跨进程文件锁和多键事务；CRC32 只能检查偶发损坏，不能防止人为重算；本报告的性能数据只在 Windows GNU 环境测量。后续可以加入 WAL 阈值触发、独占文件锁、批量事务边界和 Linux CI，但这些不影响本次课程基础要求与 B 模块提高项的验收。

第 6 章 课程设计收获及体会

6.1 对 Rust 与存储可靠性的理解

这次项目让我不再只从语法例子理解 Rust。Store::get 返回借用，SetOutcome 和 WalRecord 用枚举表达互斥状态，Result 与?把 IO、JSON 和业务错误沿调用链传递；Arc 解决共享所有权，Mutex 或 RwLock 保护可变状态。锁的范围必须覆盖一项写操作中不可拆开的 WAL 和内存步骤，而不是简单追求锁住的代码越少越好。

我最大的认识是，持久化的重点不是“能写文件”，而是“什么时候可以告诉客户端成功”。append、flush、sync_data、内存更新和响应的顺序共同决定崩溃结果。Snapshot 压缩也不能只看文件变小，还要保证处理中断时至少有一份有效恢复来源。通过主动损坏文件测试，我更清楚地理解了正常路径之外的系统设计。

6.2 接口协作、Git 与实验方法

与 A、C 交接让我认识到，个人模块完成以后还要把接口语义讲清楚。A 需要知道页面状态由哪些字段组成，C 需要知道每个存储结果和错误怎样返回；B 内部如何计算 CRC 或切换快照则不应泄漏给上层。公共接口修改前先核对调用方，分支完成后再合并和回归，比三个人同时修改同一文件更稳妥。

性能实验也让我学会不预设结论。提高版压缩前更大、普通写入稍快，都需要结合元数据和机器波动解释；只有固定输入和持久化条件，改变单一变量并保存原始样本，结果才有意义。Git 标签和独立实验分支使报告中的基础版、提高版都能找到对应代码，而不是只保留最后一次修改。

6.3 总结

项目完成了命令、客户端、服务器、内存存储、文件恢复和多客户端访问，并继续实现 Snapshot 压缩、异步并发和真实 Web 实验平台。我负责的 B 模块从离线 CRUD 发展到 Snapshot 加增量 WAL，后期又完成真实后端和实验接口接入。这个过程使我对 Rust 系统编程、存储一致性、接口交接和工程测试形成了较完整的理解。

参考文献

- [1] Steve Klabnik, Carol Nichols. The Rust Programming Language[M]. 2nd Edition. No Starch Press, 2023.
- [2] Rust Project Developers. Rust Standard Library Documentation: std::collections::BTreeMap[EB/OL].
- [3] Tokio Project. Tokio Documentation: asynchronous runtime for Rust[EB/OL].
- [4] Serde Project. Serde: Serialization Framework for Rust[EB/OL].
- [5] Tim Bray. RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format[S]. IETF, 2017.
- [6] crc32fast Contributors. crc32fast crate documentation[EB/OL].
- [7] 鹿浩. 高级系统编程技术课程要求[Z]. 河海大学计算机与软件学院, 2026.
- [8] 课程项目资料. RUST 项目步骤分解（仅供参考）[Z]. 2026.
- [9] 姚必顺. 数据层基础内容[Z]. 课程设计过程文档, 2026.
- [10] 姚必顺. 数据层提高部分[Z]. 课程设计过程文档, 2026.
- [11] 姚必顺. B 模块存储与持久化说明[Z]. 项目说明文档, 2026.
- [12] Rust_KV 项目组. Rust_KV 源代码与测试记录 [CP/OL]. https://github.com/xzh-testaccount/Rust_KV, 2026.